

XCS221 Assignment 2 — Sentiment Classification

Due Sunday, March 29 at 11:59pm PT.

Guidelines

1. If you have a question about this homework, we encourage you to post your question on our Slack channel, at <http://xcs221-scpd.slack.com/>
2. Familiarize yourself with the collaboration and honor code policy before starting work.
3. For the coding problems, you must use the packages specified in the provided environment description. Since the autograder uses this environment, we will not be able to grade any submissions which import unexpected libraries.

Submission Instructions

Written Submission: Some questions in this assignment require a written response. For these questions, you should submit a PDF with your solutions online in the online student portal. As long as the PDF is legible and organized, the course staff has no preference between a handwritten and a typeset $\text{\LaTeX}$ submission. If you wish to typeset your submission and are new to $\text{\LaTeX}$, you can get started with the following:

- Type responses only in `submission.tex`.
- Submit the compiled PDF, **not** `submission.tex`.
- Use the commented instructions within the `Makefile` and `README.md` to get started.

Also note that your answers should be in order and clearly and correctly labeled to receive credit. Be sure to submit your final answers as a PDF and **tag all pages correctly when submitting to Gradescope**.

Coding Submission: Some questions in this assignment require a coding response. For these questions, you should submit only the `src/submission.py` file in the online student portal. For further details, see Writing Code and Running the Autograder below.

Honor code

We strongly encourage students to form study groups. Students may discuss and work on homework problems in groups. However, each student must write down the solutions independently, and without referring to written notes from the joint session. In other words, each student must understand the solution well enough in order to reconstruct it by him/herself. In addition, each student should write on the problem set the set of people with whom s/he collaborated. Further, because we occasionally reuse problem set questions from previous years, we expect students not to copy, refer to, or look at the solutions in preparing their answers. It is an honor code violation to intentionally refer to a previous year's solutions. More information regarding the Stanford honor code can be found at <https://communitystandards.stanford.edu/policies-and-guidance/honor-code>.

Writing Code and Running the Autograder

All your code should be entered into `src/submission.py`. When editing `src/submission.py`, please only make changes between the lines containing `### START_CODE_HERE ###` and `### END_CODE_HERE ###`. Do not make changes to files other than `src/submission.py`.

The unit tests in `src/grader.py` (the autograder) will be used to verify a correct submission. Run the autograder locally using the following terminal command within the `src/` subdirectory:

```
$ python grader.py
```

There are two types of unit tests used by the autograder:

- **basic:** These tests are provided to make sure that your inputs and outputs are on the right track, and that the hidden evaluation tests will be able to execute.
- **hidden:** These unit tests are the evaluated elements of the assignment, and run your code with more complex inputs and corner cases. Just because your code passed the basic local tests does not necessarily mean that they will pass all of the hidden tests. These evaluative hidden tests will be run when you submit your code to the Gradescope autograder via the online student portal, and will provide feedback on how many points you have earned.

For debugging purposes, you can run a single unit test locally. For example, you can run the test case `3a-0-basic` using the following terminal command within the `src/` subdirectory:

```
$ python grader.py 3a-0-basic
```

Before beginning this course, please walk through the [Anaconda Setup for XCS Courses](#) to familiarize yourself with the coding environment. Use the env defined in `src/environment.yml` to run your code. This is the same environment used by the online autograder.

Test Cases

The autograder is a thin wrapper over the python `unittest` framework. It can be run either locally (on your computer) or remotely (on SCPD servers). The following description demonstrates what test results will look like for both local and remote execution. For the sake of example, we will consider two generic tests: `1a-0-basic` and `1a-1-hidden`.

Local Execution - Hidden Tests

All hidden tests rely on files that are not provided to students. Therefore, the tests can only be run remotely. When a hidden test like `1a-1-hidden` is executed locally, it will produce the following result:

```
----- START 1a-1-hidden: Test multiple instances of the same word in a sentence.
----- END 1a-1-hidden [took 0:00:00.011989 (max allowed 1 seconds), ???/3 points] (hidden test ungraded)
```

Local Execution - Basic Tests

When a basic test like `1a-0-basic` passes locally, the autograder will indicate success:

```
----- START 1a-0-basic: Basic test case.
----- END 1a-0-basic [took 0:00:00.000062 (max allowed 1 seconds), 2/2 points]
```

When a basic test like `1a-0-basic` fails locally, the error is printed to the terminal, along with a stack trace indicating where the error occurred:

```
----- START 1a-0-basic: Basic test case.
<class 'AssertionError'>
{'a': 2, 'b': 1} != None ← This error caused the test to fail.
File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/grader.py", line 23, in test_0
    submission.extractWordFeatures("a b a") ← In this case, start your debugging
                                           in line 23 of grader.py.
File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEqual
    assertion_func(first, second, msg=msg)
File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
----- END 1a-0-basic [took 0:00:00.003809 (max allowed 1 seconds), 0/2 points]
```

Remote Execution

Basic and hidden tests are treated the same by the remote autograder. Here are screenshots of failed basic and hidden tests. Notice that the same information (error and stack trace) is provided as the in local autograder, now for both basic and hidden tests.

1a-0-basic) Basic test case. (0.0/2.0)

```
<class 'AssertionError': {'a': 2, 'b': 1} != None
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
File "/autograder/source/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
File "/autograder/source/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
File "/autograder/source/grader.py", line 23, in test_0
    submission.extractWordFeatures("a b a"))
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEquals
    assertion_func(first, second, msg=msg)
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
```

Just like in the local autograder, this error caused the test to fail.

Just like in the local autograder, start your debugging in line 23 of grader.py.

1a-1-hidden) Test multiple instances of the same word in a sentence. (0.0/3.0)

```
<class 'AssertionError': {'a': 23, 'ab': 22, 'aa': 24, 'c': 16, 'b': 15} != None
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
File "/autograder/source/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
File "/autograder/source/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
File "/autograder/source/grader.py", line 31, in test_1
    self.compare_with_solution_or_wait(submission, 'extractWordFeatures', lambda f: f(sentence))
File "/autograder/source/graderUtil.py", line 183, in compare_with_solution_or_wait
    self.assertEqual(ans1, ans2)
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEquals
    assertion_func(first, second, msg=msg)
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
```

This error caused the test to fail.

Start your debugging in line 31 of grader.py.

Finally, here is what it looks like when basic and hidden tests pass in the remote autograder.

1a-0-basic) Basic test case. (2.0/2.0)

1a-1-hidden) Test multiple instances of the same word in a sentence. (3.0/3.0)

1. Building Intuition for Bag-of-Words Features and Linear Classification

Social media platforms like Twitter are rich sources of emotional expression. In this homework, you'll build classifiers to detect emotions in tweets using linear classification with cross-entropy loss and softmax activation. Consider the following dataset of 6 tweets, each labeled with exactly one emotion: **joy** (J), **anger** (A), or **fear** (F):

Tweet	Emotion	One-hot encoding
"amazing day"	Joy	[1, 0, 0]
"scared of spiders"	Fear	[0, 0, 1]
"love this"	Joy	[1, 0, 0]
"so angry"	Anger	[0, 1, 0]
"so so worried about tomorrow"	Fear	[0, 0, 1]
"hate waiting"	Anger	[0, 1, 0]

Each tweet x is mapped to a feature vector $f(x)$ using **bag-of-words representation** (you can think of it as word counts). Machine learning models don't directly take a string as input; instead, as you have learned, they work with matrices (like NumPy arrays and tensors). One way to convert strings to tensors is "bag-of-words" features, which treat input texts as a "bag of words" and represents them using a vector with the same length as our vocabulary. The resulting representation is a vector, where each position corresponds to a word in the vocabulary, and each value represents how many times that word appears in the input text. If we have a very large vocabulary, we would end up with a very sparse vector with many 0's.

For this problem, let's assume our vocabulary consists of all unique words in the tweets above: **{about, amazing, angry, day, hate, love, of, scared, so, spiders, this, tomorrow, waiting, worried}**.

To build our multi-class classifier, we use a weight matrix $\mathbf{W} \in \mathbb{R}^{d \times 3}$ where d is the feature dimension (vocabulary size) and 3 is the number of classes. The model computes logits as $\mathbf{z} = \mathbf{W}^T f(x)$ and applies softmax to convert logits into probability values that sum to 1. Let $\mathbf{p}$ be the 3-dimensional vector of output probabilities, and p_k be the probability that the k th element of $\mathbf{p}$ has a true label of 1. The softmax probabilities are computed in the following way:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^3 e^{z_j}}$$

Then, given the softmax probabilities, the classifier will use argmax to choose the prediction class with the highest p_k .

The cross-entropy loss measures the difference between a model's predicted probabilities and the true probability distribution of the data. For a single example (with three output classes), it can be computed as follows:

$$\text{Loss}_{\text{CE}}(x, \mathbf{y}, \mathbf{W}) = - \sum_{k=1}^3 y_k \log p_k$$

where $\mathbf{y}$ is the one-hot encoded true label vector.

(a) [0.50 points (Written)] Feature Representation

Using the vocabulary given above, write out the bag-of-words feature vector $f(x)$ for the tweet "so so worried about tomorrow".

What we expect: A feature vector corresponding to the vocabulary in alphabetical order: [about, amazing, angry, day, hate, love, of, scared, so, spiders, this, tomorrow, waiting, worried].

Feature vector: $f(x) = [1, 0, 0, 0, 0, 0, 0, 0, 2, 0, 0, 1, 0, 1]$

(b) [1 point (Written)] Softmax Computation

Given logits $\mathbf{z} = [2.0, 1.0, -1.0]$ for the three classes [Joy, Anger, Fear], compute the softmax probabilities. Show your work.

What we expect: Return the softmax probability vector $[P(\text{Joy}), P(\text{Anger}), P(\text{Fear})]$ with values rounded to 3 decimal places.

The softmax formula is:

$$P(i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Step 1: Compute the exponentials

Joy: $z = 2.0 \rightarrow e^2 = 7.389$

Anger: $z = 1.0 \rightarrow e^1 = 2.718$

Fear: $z = -1.0 \rightarrow e^{-1} = 0.368$

Step 2: Compute the sum

$$\sum e^{z_j} = 7.389 + 2.718 + 0.368 = 10.475$$

Step 3: Divide each by the sum

$$P(\text{Joy}) = \frac{7.389}{10.475} = 0.705$$

$$P(\text{Anger}) = \frac{2.718}{10.475} = 0.259$$

$$P(\text{Fear}) = \frac{0.368}{10.475} = 0.035$$

Result: $[P(\text{Joy}), P(\text{Anger}), P(\text{Fear})] = [0.705, 0.259, 0.035]$

As a sanity check, verify that the probabilities add to 1:

$$0.705 + 0.259 + 0.035 = 1.0$$

(c) [1 point (Written)] Cross-Entropy Loss

For the tweet “so angry” with true label Anger (one-hot: $[0, 1, 0]$), suppose your model outputs probabilities $[0.2, 0.7, 0.1]$. Calculate the cross-entropy loss for this example. Then explain what happens to the loss as the predicted probability for the correct class approaches 1.

What we expect:

- i. The numerical cross-entropy loss value
- ii. A brief explanation (2-3 sentences) of the loss behavior

The cross-entropy loss formula is:

$$L = - \sum_i y_i \log(p_i)$$

where y is the one-hot true label vector and p is the predicted probability vector.

Given:

- True label: $y = [0, 1, 0]$
- Predicted probabilities: $p = [0.2, 0.7, 0.1]$

Step 1: Expand the sum

$$L = -(y_{\text{Joy}} \log p_{\text{Joy}} + y_{\text{Anger}} \log p_{\text{Anger}} + y_{\text{Fear}} \log p_{\text{Fear}})$$

$$L = -(0 \cdot \log(0.2) + 1 \cdot \log(0.7) + 0 \cdot \log(0.1))$$

Step 2: Because of the one-hot encoding, only the correct class term survives

$$L = -\log(0.7) = 0.357$$

Answer:

- Cross-Entropy Loss = 0.357
- Loss Behavior

As the predicted probability for the correct class approaches 1, the loss approaches 0, since $-\log(1) = 0$. Conversely, as that probability approaches 0, the loss grows toward $+\infty$, since $-\log(x) \rightarrow \infty$ as $x \rightarrow 0$. This asymmetric behavior means the model is heavily penalized for being confidently wrong, which encourages the model to assign high probability to the true class during training.

(d) [3 points (Written)] Gradient Analysis

Derive $\frac{\partial \text{Loss}_{\text{CE}}}{\partial z_k}$, the gradient of the cross-entropy loss with respect to the logit z_k . Show the mathematical steps to find $\frac{\partial \text{Loss}_{\text{CE}}}{\partial z_k}$ and express your final answer in terms of p_k and y_k only. Then, explain intuitively why this expression makes sense for gradient-based learning.

What we expect:

- Mathematical derivation showing the steps to compute $\frac{\partial \text{Loss}_{\text{CE}}}{\partial z_k}$. (Hint: you may need to use the chain rule.)
- Final gradient expression in terms of p_k and y_k
- A brief intuitive explanation of why this gradient expression makes sense for learning (2-3 sentences)

Formulas for cross-entropy loss and softmax are:

$$\text{Loss}_{\text{CE}} = - \sum_i y_i \log(p_i) \quad \text{and} \quad p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Step 1: Apply the Chain Rule

$$\frac{\partial L}{\partial z_k} = \sum_i \frac{\partial L}{\partial p_i} \cdot \frac{\partial p_i}{\partial z_k}$$

Computing $\frac{\partial L}{\partial p_i}$ (straightforward log derivative):

$$\frac{\partial L}{\partial p_i} = -\frac{y_i}{p_i}$$

Step 2: Compute the Softmax

This requires two cases:

Case 1: $i = k$ (quotient rule, where numerator and denominator both depend on z_k):

$$\frac{\partial p_k}{\partial z_k} = \frac{e^{z_k} \sum_j e^{z_j} - e^{z_k} \cdot e^{z_k}}{\left(\sum_j e^{z_j}\right)^2} = p_k(1 - p_k)$$

Case 2: $i \neq k$ (only denominator depends on z_k):

$$\frac{\partial p_i}{\partial z_k} = \frac{0 - e^{z_i} \cdot e^{z_k}}{\left(\sum_j e^{z_j}\right)^2} = -p_i p_k$$

Step 3: Combine via Chain Rule

$$\begin{aligned} \frac{\partial L}{\partial z_k} &= \underbrace{\left(-\frac{y_k}{p_k}\right) \cdot p_k(1 - p_k)}_{i = k \text{ term}} + \underbrace{\sum_{i \neq k} \left(-\frac{y_i}{p_i}\right) \cdot (-p_i p_k)}_{i \neq k \text{ terms}} \\ &= -y_k(1 - p_k) + p_k \sum_{i \neq k} y_i \\ &= -y_k + y_k p_k + p_k \sum_{i \neq k} y_i \\ &= -y_k + p_k \underbrace{\left(y_k + \sum_{i \neq k} y_i\right)}_{=\sum_i y_i = 1 \text{ (one-hot)}} \\ &= p_k - y_k \end{aligned}$$

Answers

ii. Final Gradient Expression

$$\frac{\partial L}{\partial z_k} = p_k - y_k$$

iii. Intuitive Explanation

The gradient is simply the difference between what the model predicted and what the true label was. If the model over-estimates the probability of class k ($p_k > y_k$), the gradient is positive and pushes z_k down; if it under-estimates ($p_k < y_k$), the gradient is negative and pushes z_k up. When the model is perfectly correct ($p_k = y_k = 1$ for the true class), the gradient vanishes to zero (meaning there is nothing left to learn), which is exactly the desired behavior for gradient-based optimization.

2. Building Intuition for Embeddings and Multilayer Perceptron

In the previous problem, we used bag-of-words features for sentiment classification. It is now an outdated method of representing texts in machine learning due to many limitations. For example, it doesn't capture word relationships (e.g., "amazing" and "wonderful" are similar but treated as completely different features) and creates very sparse, high-dimensional vectors. In this problem, we'll explore how neural networks with word embeddings can address these issues.

We'll use the same set of tweets from Problem 1, but now represent each word with a dense 2-dimensional embedding vector. For simplicity, assume we have the following pre-trained word embeddings:

Word	Embedding
amazing	[0.8, 0.6]
day	[0.2, 0.1]
scared	[-0.5, -0.7]
of	[0.0, 0.0]
spiders	[-0.3, -0.4]
love	[0.9, 0.4]
this	[0.0, 0.0]
so	[0.1, 0.0]
angry	[-0.6, -0.8]
worried	[-0.3, -0.6]
about	[0.0, -0.1]
tomorrow	[0.2, -0.2]
hate	[-0.8, -0.5]
waiting	[-0.1, -0.3]

For simplicity, we will focus on **binary sentiment classification (positive or negative)** instead of three-class classification for this problem only.

(a) [1 point (Written)] Tweet Representation via Averaging

One simple approach is to represent each tweet as the average of its word embeddings. Compute the averaged embedding representation for "so angry". Comment on one advantage and one disadvantage of this approach.

What we expect:

- A 2-dimensional averaged embedding vectors
- Discuss one benefit and one limitation of creating embeddings for a text by averaging embeddings of its component words (2-3 sentences)

Averaged Embedding for "so angry"

so = [0.1, 0.0]

angry = [-0.6, -0.8]

Compute the average:

$$\begin{aligned}
 f(\text{"so angry"}) &= \frac{1}{2} ([0.1, 0.0] + [-0.6, -0.8]) \\
 &= \frac{1}{2} [0.1 + (-0.6), 0.0 + (-0.8)] \\
 &= \frac{1}{2} [-0.5, -0.8]
 \end{aligned}$$

- i. Averaged Embedding = [-0.25, -0.4]
- ii. Advantage and Disadvantage

Advantage Averaging is computationally cheap and produces a fixed-size representation regardless of tweet length. It also leverages the semantic geometry of the embedding space: tweets with semantically similar words will have similar average vectors, enabling reasonable generalization even on unseen text.

Disadvantage Averaging discards all positional and structural information, making "I am not happy" and "I am happy" potentially very similar in representation. This means the approach cannot capture negation, context, or the relative importance of individual words, which are often critical for accurate sentiment classification.

(b) [2 points (Written)] **Forward Pass**

Using the averaged representation for "so angry" from part (a), perform a forward pass step-by-step to find the predicted label $\hat{y}$. Show your intermediate calculations by filling in the table below.

For the sake of simplicity, we'll use a 2-input, 2-hidden neuron, 1-output architecture consisting of:

- i. **Input layer:** Word embeddings $\mathbf{x}$
- ii. **Hidden layer:** 2 neurons with ReLU activation, where $\text{ReLU}(\mathbf{x}) = \max(0, \mathbf{x})$
- iii. **Output layer:** 1 neuron (binary sentiment classification) with sigmoid activation

The forward pass equations are:

$$\begin{aligned}\mathbf{h} &= \text{ReLU}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) \\ z &= \mathbf{W}^{(2)}\mathbf{h} + b^{(2)} \\ \hat{y} &= \sigma(z) = \frac{1}{1 + e^{-z}}\end{aligned}$$

where $\mathbf{x} = [x_1, x_2]^T$ is the 2D input representation, here $\mathbf{x} = [x_1, x_2]^T$ is the 2D input representation, $\hat{y}$ is the predicted label, $\mathbf{W}^{(1)} \in \mathbb{R}^{2 \times 2}$, $\mathbf{b}^{(1)} \in \mathbb{R}^2$, $\mathbf{W}^{(2)} \in \mathbb{R}^{1 \times 2}$, and $b^{(2)} \in \mathbb{R}$.

Network parameters:

$$\begin{aligned}\mathbf{W}^{(1)} &= \begin{bmatrix} 1.0 & 0.5 \\ -0.5 & 1.0 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix} \\ \mathbf{W}^{(2)} &= [0.8 \quad -0.6], \quad b^{(2)} = 0.3\end{aligned}$$

You should write your answers by copying and pasting the given table and filling its cells, or using bullet points to clearly state the value of each cell.

What we expect: Compute forward pass values at each node. Show your calculations for each step by filling in the table provided.

Node/Variable	Formula/Computation	Value
$\mathbf{x}$	NA	
$\mathbf{h}$		
z		
$\hat{y}$		

Forward Pass for "so angry"

The averaged embedding for "so angry" is: $x = \begin{bmatrix} -0.25 \\ -0.4 \end{bmatrix}$

Step 1: Pre-activation of Hidden Layer

$$W^{(1)}x + b^{(1)} = \begin{bmatrix} 1.0 & 0.5 \\ -0.5 & 1.0 \end{bmatrix} \begin{bmatrix} -0.25 \\ -0.4 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}$$

Row 1: $1.0(-0.25) + 0.5(-0.4) + 0.1 = -0.25 - 0.20 + 0.1 = -0.35$

Row 2: $-0.5(-0.25) + 1.0(-0.4) + (-0.2) = 0.125 - 0.40 - 0.2 = -0.475$

$$W^{(1)}x + b^{(1)} = \begin{bmatrix} -0.35 \\ -0.475 \end{bmatrix}$$

Step 2: Apply ReLU Activation

$$h = \text{ReLU} \begin{bmatrix} -0.35 \\ -0.475 \end{bmatrix} = \begin{bmatrix} \max(0, -0.35) \\ \max(0, -0.475) \end{bmatrix} = \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix}$$

Both values are negative, so ReLU clamps them both to 0.

Step 3: Output Pre-activation

$$z = W^{(2)}h + b^{(2)} = \begin{bmatrix} 0.8 & -0.6 \end{bmatrix} \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix} + 0.3$$

$$z = 0.8(0.0) + (-0.6)(0.0) + 0.3 = \mathbf{0.3}$$

Step 4: Apply Sigmoid Activation

$$\hat{y} = \sigma(0.3) = \frac{1}{1 + e^{-0.3}} = \frac{1}{1 + 0.741} = \frac{1}{1.741} \approx \mathbf{0.574}$$

Summary of Forward Pass

Node/Variable	Formula/Computation	Value
$\mathbf{x}$	NA	$[-0.25, -0.4]^T$
$\mathbf{h}$	$\text{ReLU}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$	$[0.0, 0.0]^T$
z	$\mathbf{W}^{(2)}\mathbf{h} + b^{(2)}$	0.3
$\hat{y}$	$\sigma(z) = \frac{1}{1+e^{-z}}$	0.574

Since $\hat{y} = 0.574 > 0.5$, the model predicts positive sentiment (which is incorrect for "so angry"). This is largely because the strongly negative embeddings caused both ReLU neurons to die (output 0), losing all input signal and leaving only the bias $b^2 = 0.3$ to drive the prediction.

(c) [6 points (Written)] Backpropagation

Using the same weights, bias and input vector as in part (b), first compute the loss value L . Then, perform backpropagation to compute gradients $\frac{\partial L}{\partial \mathbf{W}^{(1)}}$, $\frac{\partial L}{\partial \mathbf{b}^{(1)}}$, $\frac{\partial L}{\partial \mathbf{W}^{(2)}}$, and $\frac{\partial L}{\partial b^{(2)}}$. Assume the true label is $y_{\text{true}} = 0$ (negative sentiment) and we're using binary cross-entropy loss:

$$L = -[y_{\text{true}} \log(\hat{y}) + (1 - y_{\text{true}}) \log(1 - \hat{y})]$$

You should write your answers by copying and pasting the tables below and filling their cells, or using bullet points to clearly state the value of each cell.

What we expect:

Variable	Formula/Computation	Value
L		

Gradient	Formula (Chain Rule)	Evaluated Value
$\frac{\partial L}{\partial \hat{y}}$		
$\frac{\partial L}{\partial \tilde{z}}$		
$\frac{\partial L}{\partial \mathbf{h}}$		
$\frac{\partial L}{\partial \mathbf{W}^{(2)}}$		
$\frac{\partial L}{\partial \mathbf{b}^{(2)}}$		
$\frac{\partial L}{\partial \mathbf{W}^{(1)}}$		
$\frac{\partial L}{\partial \mathbf{b}^{(1)}}$		

- i. Compute the loss value L
- ii. Starting from $\frac{\partial L}{\partial \hat{y}}$, compute gradients for each node working backwards and fill out the tables given above. For the formula column, you should express each partial in terms of partials above it in the table (recall backpropagation from lecture!)
- iii. Feel free to reuse values you already computed in part b

Backpropagation for "so angry"

Reusing values from the forward pass:

$$\begin{aligned}
 x &= [-0.25, -0.4]^T \\
 pre - activation &= [-0.35, -0.475]^T \\
 h &= [0.0, 0.0]^T \\
 z &= 0.3 \\
 \hat{y} &= 0.574 \\
 y_{\text{true}} &= 0
 \end{aligned}$$

i. Computing Loss L

Binary cross-entropy loss:

$$L = -[y_{\text{true}} \log(\hat{y}) + (1 - y_{\text{true}}) \log(1 - \hat{y})]$$

$$L = -[0 \cdot \log(0.574) + 1 \cdot \log(1 - 0.574)]$$

$$L = -\log(0.426) = \mathbf{0.853}$$

ii. Backward Pass

Gradient of loss w.r.t. prediction

$$\frac{\partial L}{\partial \hat{y}} = - \left[\frac{y_{\text{true}}}{\hat{y}} - \frac{1 - y_{\text{true}}}{1 - \hat{y}} \right] = \frac{1}{1 - \hat{y}} = \frac{1}{1 - 0.574} = \frac{1}{0.426} \approx \mathbf{2.347}$$

Gradient w.r.t. sigmoid

$$\begin{aligned}\frac{\partial L}{\partial z} &= \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} = \frac{1}{1 - \hat{y}} \cdot \hat{y}(1 - \hat{y}) = \hat{y} - y_{\text{true}} \\ &= 0.574 - 0 = \mathbf{0.574}\end{aligned}$$

Gradient w.r.t. output weights

$$\frac{\partial L}{\partial W^{(2)}} = \frac{\partial L}{\partial z} \cdot h^T = 0.574 \cdot [0.0, 0.0] = [\mathbf{0.0}, \mathbf{0.0}]$$

Gradient w.r.t. output bias

$$\frac{\partial L}{\partial b^{(2)}} = \frac{\partial L}{\partial z} \cdot 1 = \mathbf{0.574}$$

Gradient w.r.t. hidden activations

$$\frac{\partial L}{\partial h} = \frac{\partial L}{\partial z} \cdot W^{(2)T} = 0.574 \cdot \begin{bmatrix} 0.8 \\ -0.6 \end{bmatrix} = \begin{bmatrix} 0.459 \\ -0.344 \end{bmatrix}$$

Gradient w.r.t. pre-activation

$$\frac{\partial L}{\partial \text{pre-act}} = \frac{\partial L}{\partial h} \odot \text{ReLU}'(\text{pre-act})$$

Since pre-activations were $[-0.35, -0.475]$, both negative, $\text{ReLU}' = 0$ for both neurons:

$$= \begin{bmatrix} 0.459 \\ -0.344 \end{bmatrix} \odot \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix}$$

Gradient w.r.t. hidden weights

$$\frac{\partial L}{\partial W^{(1)}} = \frac{\partial L}{\partial \text{pre-act}} \cdot x^T = \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix} \begin{bmatrix} -0.25 & -0.4 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

Gradient w.r.t. hidden bias

$$\frac{\partial L}{\partial b^{(1)}} = \frac{\partial L}{\partial \text{pre-act}} = [\mathbf{0.0}, \mathbf{0.0}]^T$$

Summary Table

Variable	Formula/Computation	Value
L	$-[y_{\text{true}} \log(\hat{y}) + (1 - y_{\text{true}}) \log(1 - \hat{y})]$	0.853

(d) [0.50 points (Written)] Embedding Analysis

Looking at the provided word embeddings, identify patterns in how positive emotion words, negative emotion words, and neutral words are positioned in the 2D embedding space. You don't have to plot anything for this problem; simply observe the patterns. What does this suggest about how word embeddings capture semantic relationships?

What we expect:

Gradient	Formula (Chain Rule)	Evaluated Value
$\frac{\partial L}{\partial \hat{y}}$	$-\left[\frac{y_{\text{true}}}{\hat{y}} - \frac{1-y_{\text{true}}}{1-\hat{y}}\right]$	2.347
$\frac{\partial L}{\partial z}$	$\hat{y} - y_{\text{true}}$	0.574
$\frac{\partial L}{\partial \mathbf{h}}$	$\frac{\partial L}{\partial z} \cdot W^{(2)T}$	$[0.459, -0.344]^T$
$\frac{\partial L}{\partial \mathbf{W}^{(2)}}$	$\frac{\partial L}{\partial z} \cdot \mathbf{h}^T$	$[0.0, 0.0]$
$\frac{\partial L}{\partial b^{(2)}}$	$\frac{\partial L}{\partial z} \cdot 1$	0.574
$\frac{\partial L}{\partial \mathbf{W}^{(1)}}$	$\frac{\partial L}{\partial \text{pre-act}} \cdot x^T$	$[[0, 0], [0, 0]]$
$\frac{\partial L}{\partial \mathbf{b}^{(1)}}$	$\frac{\partial L}{\partial \text{pre-act}}$	$[0.0, 0.0]^T$

- Describe the spatial clustering of different word types (1-2 sentences)
- Explain how embeddings capture semantic similarity and its advantage over bag-of-words (1-2 sentences)

i. Spatial Clustering

The embeddings reveal clear geometric clustering by sentiment: positive words ("amazing", "love") cluster in the upper-right quadrant with large positive values on both dimensions, while negative emotion words ("angry", "hate", "scared", "worried") cluster in the lower-left quadrant with consistently negative values. Neutral and function words ("of", "this", "so") cluster near the origin $[0, 0]$, reflecting their lack of emotional content.

ii. Semantic Similarity and Advantage over Bag-of-Words

Word embeddings encode semantic similarity as geometric proximity: words with similar meaning or emotional sentiment are placed close together in vector space, meaning operations like distance and direction carry real language meaning. Unlike bag-of-words, which treats every word as an independent, orthogonal dimension (so "love" and "amazing" share no information), embeddings allow the model to generalize across synonyms and related words: a model that learns "hate" is negative automatically gains partial knowledge about "angry" simply because they occupy similar regions of the embedding space.

3. Linear Classifier

Now, we will implement the first classifier that can classify tweet sentiments. Recall that in our data, each tweet is labeled with one of three emotions (**joy**, **anger**, or **fear**) in one-hot encoding. For example:

Text	One-hot label
i feel very happy and excited since i learned so many things	[1,0,0]
i feel angered and firey	[0,1,0]
i remember feeling acutely distressed for a few days	[0,0,1]

(a) [2 points (Coding)] **Build vocabulary**

Implement `build_vocabulary(examples)`, which builds a vocabulary using words seen in the training examples. Your implementation should populate the `Vocabulary` class provided in `util.py`, which will be used in part b to build a sparse feature representation for any input text.

What we expect: Implement `build_vocabulary(examples)` using the `Vocabulary` class from `util.py`.

(b) [2 points (Coding)] **Text to Features**

Implement `text_to_features(text, vocab)`, which converts a text string into a sparse feature vector. Reference 1a if you want to revisit the steps for this process.

(c) [2 points (Coding)] **NumPy Softmax**

Implement `numpy_softmax(logits)` using `einops` and NumPy library functions only.

(d) [2 points (Coding)] **NumPy Cross-Entropy Loss**

Implement `numpy_cross_entropy_loss(predictions, targets, epsilon)`, which returns a floating point number measuring how far the predicted probabilities are from the true one-hot labels, on average.

(e) [2 points (Coding)] **NumPy Gradient Computation**

Implement `numpy_compute_gradients(features, predictions, targets)`, which returns the gradients for weight and bias, respectively. This is later used to update model weights using backpropagation.

(f) [2 points (Coding)] **Linear classifier predictor**

Implement `predict_linear_classifier(features, labels, weights, bias)`, which makes predictions on input features using trained weights and bias, and computes the accuracy of the predictions.

Your implementation should include:

- Compute logits by multiplying features with weights and adding bias
- Apply softmax to convert logits to probabilities
- Get predicted class labels by taking the argmax of probabilities
- Compare with true labels to compute accuracy

(g) [5 points (Coding)] **Train linear classifier**

Implement `train_linear_classifier(...)` which integrates all previous components into a complete training loop.

Your implementation should:

- Initialize weights randomly and bias as a 0-vector
- For each epoch: compute forward pass, calculate loss, compute gradients, and update parameters correspondingly. At the end of the epoch, print out the training loss and validation accuracy
- Use your implemented softmax, cross-entropy loss, gradient, and predictor functions
- You should train the model on training data only; **do not** train the model on validation data.

Finally, run `python submission.py --model linear`. The autograder requires that you achieve at least a 0.4 accuracy using default hyperparameters.

(h) [0.50 points (Written)] **Writeup**

If you adjust `lr`, the learning rate, how do you expect loss and validation accuracy to change during training? Run the terminal command `python submission.py --model linear --lr [your_learning_rate]` to experiment with three different learning rates. Report what they are and what training behaviors you observed. Why did the learning rate affect loss and accuracy this way?

For example, you can run: `python submission.py --model linear --lr 0.1`. If you don't include an `--lr` argument, the default learning rate is 0.2.

What we expect: In 2-3 sentences, report three different `lr`'s you experimented with, and describe how they affected training behaviors. Explain why the learning rate affects loss and accuracy this way.

If the learning rate is too small (e.g., `lr = 0.0001`), the loss decreases very slowly (tiny steps mean many epochs needed to converge) and accuracy barely changes.

If the learning rate is too large (e.g., `lr = 100.0`), the loss explodes (overshoots the minimum, bouncing around or diverging entirely). Accuracy becomes zero as the weights become too large.

If the learning rate is just right (e.g., `lr = 0.1`), the loss decreases steadily and accuracy improves consistently until it reaches the max value of 1.0.

4. Multilayer Perceptron and Embeddings

Linear classification was a simple attempt at detecting tweet sentiments. In class, we learned that word embeddings and neural networks are powerful tools in natural language processing, and we will now implement them.

To build embeddings, we convert each word into a small, dense "embedding" vector (such as a 32-dimensional array of numbers) that captures the word's meaning. To obtain the embedding for each text, we simply average all the word embeddings to get one fixed-size vector per document - so a 500-word document and a 10-word document both become the same size vector, making them perfect inputs for your multilayer perceptron classifier while preserving much more semantic information than word counts alone.

We will implement a simple **2-layer neural network** consisting of:

- An input layer that takes averaged embeddings of size `embedding_dim`
- A hidden layer that uses ReLU activation
- An output layer that produces raw scores (logits) for each sentiment class

Note that this is different from the architecture described in problem 2.

(a) [2 points (Coding)] Text to average embedding

Implement `text_to_average_embedding(text, vocab, embedding_layer)`, which converts a text string into a single embedding vector by averaging word embeddings. This creates a fixed-size representation for variable-length tweets.

Your implementation should include:

- Create a list of indices, with each position corresponding to the vocabulary index of the words in the input text.
- Create a PyTorch tensor from the list of indices (hint: use `dtype=torch.long`).
- Pass the tensor of indices through the embedding layer to get word embeddings
- Average the embeddings across the word dimension to get a single vector (hint: use `einops`)
- Return the averaged embedding as a tensor

(b) [2 points (Coding)] Extract averaged features

Implement `extract_averaged_features(texts, vocab, embedding_layer)`, which processes a list of text strings and converts them all into a single tensor of averaged embeddings. This function builds on your `text_to_average_embedding` implementation to handle entire datasets at once.

Your implementation should:

- Initialize an empty list to collect the embedding vectors
- Loop through each text string in the input list
- For each text, call your `text_to_average_embedding` function to get its averaged embedding
- Append each averaged embedding to your collection list
- Combine all individual embeddings into a single tensor
- Return a tensor of shape `(num_texts, embedding_dim)` where each row is one text's embedding

Hint: This function essentially applies your single-text embedding function to every text in a batch, then organizes the results into the matrix format that your MLP classifier expects.

(c) [2 points (Coding)] MLP classifier Complete the `MLPClassifier` class.

(d) [2 points (Coding)] Utility functions

Implement the helper functions tailored to our three-class prediction task.

- `torch_softmax(logits)`: Convert logits to probabilities
- `torch_cross_entropy_loss(predictions, targets)`: A custom function that computes the average cross entropy loss of each training batch
- `update_parameter(param, grad, lr)`: Manual gradient descent update

(e) [2 points (Coding)] MLP predictor function

Implement `predict_mlp(texts, labels, classifier, embedding_layer, vocab)`, which evaluates a trained model on new text data.

Your implementation should:

- Set models to evaluation mode (no training)
- Extract averaged embeddings for the input texts
- Pass embeddings through the classifier to get predictions
- Compare predictions with true labels to compute accuracy

(f) [5 points (Coding)] **Train MLP classifier**

Implement `train_mlp_classifier(...)`.

This is the main training function that integrates everything together. Your implementation should:

- Build vocabulary from training texts
- Create and initialize embedding layer and MLP classifier
- Set up training loop with batch processing
- For each epoch: extract features, forward pass, compute loss, backpropagation, update parameters. At the start of each epoch, you should shuffle training data before processing them in batches. At the end of the epoch, print out the training loss and validation accuracy
- Use stochastic gradient descent to update all parameters; **do not** use optimizers (if you don't know what they are, that's perfectly fine!)
- You should train the model on training data only; **do not** train the model on validation data.

You can test your implementation with this terminal command: `python submission.py --model mlp`. The autograder requires that you achieve at least a 0.55 accuracy using default hyperparameters.

5. Product Ethics Exploration: Usage, Audience, and Safety

You will choose a modern AI product to analyze throughout the assignments and gain initial access to it.

Suggested resources:

- Pick an AI-powered product or tool (not just a raw LLM chatbot) that is accessible to you. Good candidates include: Cursor, Nano Banana, ElevenLabs, Perplexity, Claude Code, etc. Free trials or student-accessible tools are recommended.
- The product’s official website and onboarding/getting started pages
- Privacy policy, Terms of Service, Acceptable Use Policy
- App store listings, GitHub documentation or READMEs (if open-source or developer-targeted)
- Recent news coverage (e.g., TechCrunch, The Verge, Wired, NYT technology section)
- Reddit communities, Hacker News, or user forums for first impressions and experiences

In this part: You will investigate who uses the product, how it’s intended to be used, and what safeguards exist against misuse.

Why it’s important: Understanding the gap between intended and actual use, as well as potential safety risks, helps identify where AI systems can cause harm despite good intentions.

Suggested resources:

- Company “About,” “Press,” or “Investors” pages for user numbers
- Third-party analytics sites (e.g., SimilarWeb, Statista) for user data
- Case studies or testimonials published by the company
- Safety guidelines or misuse policies
- Blog posts about misuse mitigation

(a) [0.50 points (Written)] Product User Base and Demographics

- Research and report any available data on the product’s users: total number of users, historical growth, and demographic breakdowns such as user type, geography, and age group. Cite your sources (e.g., company “About,” “Press,” or “Investors” pages, third-party analytics sites).

What we expect: Provide a 2-3 sentence summary of the product’s user numbers and user composition, with cited sources.

Claude Code: Product User Base and Demographics

Claude (including Claude Code) does not have a fully transparent, product-specific user breakdown, but multiple sources provide a clear overall picture of its user base and demographics.

As of 2025–2026, Claude has roughly 18–30 million monthly active users, with rapid growth (reportedly ~1 million new users per day in early 2026) and strong enterprise adoption across 300,000+ business customers ([Backlinko, 2026](#)). Usage is globally distributed across 150+ countries, led by the U.S. and India, and skewed toward work and coding tasks, with developers and knowledge workers forming a major segment ([GetPanto, 2026](#)). Demographically, users skew young (about 50%+ aged 18–24) and male-dominated (~77%), with a mix of work (~46%), study (~19%), and personal use (~35%) ([DemandSage, 2026](#)).

Overall, while Claude Code–specific user counts are not disclosed, Anthropic reports ~300% growth in its developer user base in 2025, indicating rapid adoption among engineers and enterprise teams ([The New Stack, 2025](#)).

(b) [0.50 points (Written)] Intended Use Cases and Audience

- Summarize what you learn about the intended use cases for this product. Who are the primary target users? Reference product documentation, official resources, or company case studies/testimonials as appropriate.

What we expect: In 2-3 sentences, identify and concisely describe the main intended uses and user groups for the product.

Claude Code: Intended Use Cases and Audience

Claude Code is designed primarily as an AI-powered coding assistant that enables users to write, refactor, debug, and manage software projects through natural language, often by executing commands directly in the terminal and interacting with entire codebases ([Anthropic GitHub Docs, 2026](#); [Codecademy, 2026](#)). Its core use case is to delegate substantial engineering tasks - such as code generation, documentation, and workflow automation - to an agentic AI system that can operate across files and tools ([DataCamp, 2026](#)).

The primary target users are software developers, engineers, and technical teams, though the tool is increasingly used by adjacent roles (e.g., designers and product builders) to accelerate prototyping and development workflows ([Anthropic GitHub Docs, 2026](#); [DataCamp, 2026](#)).

(c) [0.50 points (Written)] Market Share and Competitors

- Determine the current market share of this product compared to key competitors. If exact figures are unavailable, provide best available estimates or relevant data/quotes from news or analytics sources.

What we expect: In 3-4 sentences, include a comparative overview of the product's market position, citing sources or providing reasonable estimates.

Claude Code: Market Share and Competitors

Claude Code operates in the rapidly growing AI coding assistants market, where competition is led by tools such as GitHub Copilot, OpenAI Codex, Cursor, and Google Gemini. While OpenAI still dominates overall AI assistant usage, the coding segment is more fragmented, with estimates suggesting Claude-based tools hold roughly 24–32% market share, comparable to Copilot and Cursor in a three-way tie at the top ([LinkedIn Analysis, 2025](#); [ITiger, 2025](#)).

Some reports indicate even stronger positioning in code generation specifically, with Claude models capturing up to ~42% share among developers, significantly ahead of OpenAI's ~21% ([CB Insights, 2025](#); [Yahoo Finance, 2025](#)). However, broader developer surveys still show tools like ChatGPT and Copilot leading in overall adoption, indicating that Claude Code is a top-tier but not dominant player in a fast-evolving and highly competitive market ([CodeSignal, 2025](#)).

Overall, Claude Code is positioned as a leading contender in the coding-specific AI segment, particularly strong in enterprise and advanced code-generation workflows, but faces intense competition from incumbents and rapidly growing startups in a fragmented market.

(d) [0.50 points (Written)] Misuse Risks and Company Safeguards

- Discuss at least one way the product could be misused. What steps, if any, does the company take to safeguard against this misuse? Reference any published safety guidelines, usage policies, or blog posts about misuse mitigation.
- Consider the concept of **dual use technology**: AI systems can be used for both beneficial and harmful purposes, making governance challenging. How does your chosen product exemplify this dual-use nature? Can you identify both a beneficial use case and a potential harmful misuse of the same underlying technology?

What we expect: In 4-6 sentences, identify a realistic misuse scenario and summarize the company's corresponding safety policies or interventions. Additionally, discuss how the product demonstrates dual-use characteristics, providing examples of both beneficial and potentially harmful applications.

Claude Code: Misuse Risks and Company Safeguards

Claude Code presents a clear misuse risk in that it can be used to generate or modify malicious software, such as automating the creation of exploits, phishing scripts, or obfuscated code that bypasses security controls. Anthropic addresses this through its Responsible Scaling Policy and usage guidelines, which explicitly restrict assistance for malware development, require safety filtering of harmful outputs, and incorporate monitoring and rate-limiting for suspicious usage patterns ([Anthropic, 2024](#); [Anthropic Acceptable Use Policy, 2025](#)). Additionally, Claude models

are trained with safety alignment techniques to refuse or redirect harmful requests, particularly those involving cybersecurity abuse or unauthorized system access (Anthropic, 2023).

This highlights the dual-use nature of Claude Code: the same capability that enables developers to efficiently write, debug, and secure code can also be repurposed to generate harmful or deceptive software. For example, a beneficial use case is automating code reviews and improving security practices, while a harmful misuse would be accelerating the development of malware or social engineering tools. This duality makes governance challenging, requiring continuous safeguards, policy enforcement, and model-level alignment to balance innovation with risk mitigation.

(e) [0.50 points (Written)] **Gaps and Unintended Use Cases**

- Are there any aspects of user information or misuse that appear **not** to be safeguarded against—based on what you can find published by the company or in trusted third-party reports? Briefly discuss.
- Provide at least one example of an unintended use case of the product—something the company likely did **not** intend, but that users or communities have pursued anyway. Cite examples if you find them.

What we expect: In 2-3 sentences, briefly discuss gaps in company safeguards and highlight at least one notable unintended use case, with references to sources.

Claude Code: Gaps and Unintended Use Cases

Despite existing safeguards, gaps remain in protecting against prompt injection and data exfiltration, where Claude Code can be induced to reveal sensitive files (e.g., API keys) or ignore ignore-rules like `.claudeignore`, indicating incomplete enforcement of access controls (The Register, 2026). Additionally, real-world misuse cases show that attackers have leveraged Claude to assist in cybercrime activities such as phishing, malware generation, and even large-scale extortion schemes, demonstrating that safeguards can be bypassed in practice (Reuters, 2025; LessWrong, 2025).

A notable unintended use case is the use of Claude Code as an automated hacking or extortion assistant, where users repurpose its coding and reasoning abilities to scale attacks against organizations - an outcome clearly outside Anthropic's intended productivity - focused design. These examples highlight that, while safeguards exist, agentic coding tools still introduce new attack surfaces and failure modes that are not fully mitigated by current policies or technical controls.

This handout includes space for every question that requires a written response. Please feel free to use it to handwrite your solutions (legibly, please). If you choose to typeset your solutions, the `README.md` for this assignment includes instructions to regenerate this handout with your typeset L^AT_EX solutions.

1.a

Feature vector: $f(x) = [1, 0, 0, 0, 0, 0, 0, 0, 2, 0, 0, 1, 0, 1]$

1.b

The softmax formula is:

$$P(i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Step 1: Compute the exponentials

Joy: $z = 2.0 \rightarrow e^2 = 7.389$

Anger: $z = 1.0 \rightarrow e^1 = 2.718$

Fear: $z = -1.0 \rightarrow e^{-1} = 0.368$

Step 2: Compute the sum

$$\sum e^{z_j} = 7.389 + 2.718 + 0.368 = 10.475$$

Step 3: Divide each by the sum

$$P(\text{Joy}) = \frac{7.389}{10.475} = 0.705$$

$$P(\text{Anger}) = \frac{2.718}{10.475} = 0.259$$

$$P(\text{Fear}) = \frac{0.368}{10.475} = 0.035$$

Result: $[P(\text{Joy}), P(\text{Anger}), P(\text{Fear})] = [0.705, 0.259, 0.035]$

As a sanity check, verify that the probabilities add to 1:

$$0.705 + 0.259 + 0.035 = 1.0$$

1.c

The cross-entropy loss formula is:

$$L = - \sum_i y_i \log(p_i)$$

where y is the one-hot true label vector and p is the predicted probability vector.

Given:

- True label: $y = [0, 1, 0]$
- Predicted probabilities: $p = [0.2, 0.7, 0.1]$

Step 1: Expand the sum

$$L = -(y_{\text{Joy}} \log p_{\text{Joy}} + y_{\text{Anger}} \log p_{\text{Anger}} + y_{\text{Fear}} \log p_{\text{Fear}})$$

$$L = -(0 \cdot \log(0.2) + 1 \cdot \log(0.7) + 0 \cdot \log(0.1))$$

Step 2: Because of the one-hot encoding, only the correct class term survives

$$L = -\log(0.7) = 0.357$$

Answer:

- i. Cross-Entropy Loss = 0.357
- ii. Loss Behavior

As the predicted probability for the correct class approaches 1, the loss approaches 0, since $-\log(1) = 0$. Conversely, as that probability approaches 0, the loss grows toward $+\infty$, since $-\log(x) \rightarrow \infty$ as $x \rightarrow 0$. This asymmetric behavior means the model is heavily penalized for being confidently wrong, which encourages the model to assign high probability to the true class during training.

1.d

Formulas for cross-entropy loss and softmax are:

$$\text{Loss}_{CE} = - \sum_i y_i \log(p_i) \quad \text{and} \quad p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Step 1: Apply the Chain Rule

$$\frac{\partial L}{\partial z_k} = \sum_i \frac{\partial L}{\partial p_i} \cdot \frac{\partial p_i}{\partial z_k}$$

Computing $\frac{\partial L}{\partial p_i}$ (straightforward log derivative):

$$\frac{\partial L}{\partial p_i} = -\frac{y_i}{p_i}$$

Step 2: Compute the Softmax

This requires two cases:

Case 1: $i = k$ (quotient rule, where numerator and denominator both depend on z_k):

$$\frac{\partial p_k}{\partial z_k} = \frac{e^{z_k} \sum_j e^{z_j} - e^{z_k} \cdot e^{z_k}}{\left(\sum_j e^{z_j}\right)^2} = p_k (1 - p_k)$$

Case 2: $i \neq k$ (only denominator depends on z_k):

$$\frac{\partial p_i}{\partial z_k} = \frac{0 - e^{z_i} \cdot e^{z_k}}{\left(\sum_j e^{z_j}\right)^2} = -p_i p_k$$

Step 3: Combine via Chain Rule

$$\begin{aligned} \frac{\partial L}{\partial z_k} &= \underbrace{\left(-\frac{y_k}{p_k}\right) \cdot p_k (1 - p_k)}_{i = k \text{ term}} + \underbrace{\sum_{i \neq k} \left(-\frac{y_i}{p_i}\right) \cdot (-p_i p_k)}_{i \neq k \text{ terms}} \\ &= -y_k (1 - p_k) + p_k \sum_{i \neq k} y_i \\ &= -y_k + y_k p_k + p_k \sum_{i \neq k} y_i \\ &= -y_k + p_k \underbrace{\left(y_k + \sum_{i \neq k} y_i\right)}_{=\sum_i y_i = 1 \text{ (one-hot)}} \\ &= p_k - y_k \end{aligned}$$

Answers

ii. Final Gradient Expression

$$\frac{\partial L}{\partial z_k} = p_k - y_k$$

iii. Intuitive Explanation

The gradient is simply the difference between what the model predicted and what the true label was. If the model over-estimates the probability of class k ($p_k > y_k$), the gradient is positive and pushes z_k down; if it under-estimates ($p_k < y_k$), the gradient is negative and pushes z_k up. When the model is perfectly correct ($p_k = y_k = 1$ for the true class), the gradient vanishes to zero (meaning there is nothing left to learn), which is exactly the desired behavior for gradient-based optimization.

2.a

Averaged Embedding for "so angry"

so = [0.1, 0.0]

angry = [-0.6, -0.8]

Compute the average:

$$\begin{aligned}
 f(\text{"so angry"}) &= \frac{1}{2} ([0.1, 0.0] + [-0.6, -0.8]) \\
 &= \frac{1}{2} [0.1 + (-0.6), 0.0 + (-0.8)] \\
 &= \frac{1}{2} [-0.5, -0.8]
 \end{aligned}$$

i. Averaged Embedding = [-0.25, -0.4]

ii. Advantage and Disadvantage

Advantage Averaging is computationally cheap and produces a fixed-size representation regardless of tweet length. It also leverages the semantic geometry of the embedding space: tweets with semantically similar words will have similar average vectors, enabling reasonable generalization even on unseen text.

Disadvantage Averaging discards all positional and structural information, making "I am not happy" and "I am happy" potentially very similar in representation. This means the approach cannot capture negation, context, or the relative importance of individual words, which are often critical for accurate sentiment classification.

2.b

Forward Pass for "so angry"

The averaged embedding for "so angry" is: $x = \begin{bmatrix} -0.25 \\ -0.4 \end{bmatrix}$

Step 1: Pre-activation of Hidden Layer

$$W^{(1)}x + b^{(1)} = \begin{bmatrix} 1.0 & 0.5 \\ -0.5 & 1.0 \end{bmatrix} \begin{bmatrix} -0.25 \\ -0.4 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}$$

Row 1: $1.0(-0.25) + 0.5(-0.4) + 0.1 = -0.25 - 0.20 + 0.1 = -0.35$ Row 2: $-0.5(-0.25) + 1.0(-0.4) + (-0.2) = 0.125 - 0.40 - 0.2 = -0.475$

$$W^{(1)}x + b^{(1)} = \begin{bmatrix} -0.35 \\ -0.475 \end{bmatrix}$$

Step 2: Apply ReLU Activation

$$h = \text{ReLU} \begin{bmatrix} -0.35 \\ -0.475 \end{bmatrix} = \begin{bmatrix} \max(0, -0.35) \\ \max(0, -0.475) \end{bmatrix} = \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix}$$

Both values are negative, so ReLU clamps them both to 0.

Step 3: Output Pre-activation

$$z = W^{(2)}h + b^{(2)} = \begin{bmatrix} 0.8 & -0.6 \end{bmatrix} \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix} + 0.3$$

$$z = 0.8(0.0) + (-0.6)(0.0) + 0.3 = \mathbf{0.3}$$

Step 4: Apply Sigmoid Activation

$$\hat{y} = \sigma(0.3) = \frac{1}{1 + e^{-0.3}} = \frac{1}{1 + 0.741} = \frac{1}{1.741} \approx \mathbf{0.574}$$

Summary of Forward Pass

Node/Variable	Formula/Computation	Value
$\mathbf{x}$	NA	$[-0.25, -0.4]^T$
$\mathbf{h}$	$\text{ReLU}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$	$[0.0, 0.0]^T$
z	$\mathbf{W}^{(2)}\mathbf{h} + b^{(2)}$	0.3
$\hat{y}$	$\sigma(z) = \frac{1}{1+e^{-z}}$	0.574

Since $\hat{y} = 0.574 > 0.5$, the model predicts positive sentiment (which is incorrect for "so angry"). This is largely because the strongly negative embeddings caused both ReLU neurons to die (output 0), losing all input signal and leaving only the bias $b^2 = 0.3$ to drive the prediction.

2.c

Backpropagation for "so angry"

Reusing values from the forward pass:

$$\begin{aligned} x &= [-0.25, -0.4]^T \\ pre-activation &= [-0.35, -0.475]^T \\ h &= [0.0, 0.0]^T \\ z &= 0.3 \\ \hat{y} &= 0.574 \\ y_{\text{true}} &= 0 \end{aligned}$$

i. Computing Loss L

Binary cross-entropy loss:

$$L = -[y_{\text{true}} \log(\hat{y}) + (1 - y_{\text{true}}) \log(1 - \hat{y})]$$

$$L = -[0 \cdot \log(0.574) + 1 \cdot \log(1 - 0.574)]$$

$$L = -\log(0.426) = \mathbf{0.853}$$

ii. Backward Pass

Gradient of loss w.r.t. prediction

$$\frac{\partial L}{\partial \hat{y}} = - \left[\frac{y_{\text{true}}}{\hat{y}} - \frac{1 - y_{\text{true}}}{1 - \hat{y}} \right] = \frac{1}{1 - \hat{y}} = \frac{1}{1 - 0.574} = \frac{1}{0.426} \approx \mathbf{2.347}$$

Gradient w.r.t. sigmoid

$$\begin{aligned} \frac{\partial L}{\partial z} &= \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} = \frac{1}{1 - \hat{y}} \cdot \hat{y}(1 - \hat{y}) = \hat{y} - y_{\text{true}} \\ &= 0.574 - 0 = \mathbf{0.574} \end{aligned}$$

Gradient w.r.t. output weights

$$\frac{\partial L}{\partial W^{(2)}} = \frac{\partial L}{\partial z} \cdot h^T = 0.574 \cdot [0.0, 0.0] = [\mathbf{0.0}, \mathbf{0.0}]$$

Gradient w.r.t. output bias

$$\frac{\partial L}{\partial b^{(2)}} = \frac{\partial L}{\partial z} \cdot 1 = \mathbf{0.574}$$

Gradient w.r.t. hidden activations

$$\frac{\partial L}{\partial h} = \frac{\partial L}{\partial z} \cdot W^{(2)T} = 0.574 \cdot \begin{bmatrix} 0.8 \\ -0.6 \end{bmatrix} = \begin{bmatrix} 0.459 \\ -0.344 \end{bmatrix}$$

Gradient w.r.t. pre-activation

$$\frac{\partial L}{\partial \text{pre-act}} = \frac{\partial L}{\partial h} \odot \text{ReLU}'(\text{pre-act})$$

Since pre-activations were $[-0.35, -0.475]$, both negative, $\text{ReLU}' = 0$ for both neurons:

$$= \begin{bmatrix} 0.459 \\ -0.344 \end{bmatrix} \odot \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix}$$

Gradient w.r.t. hidden weights

$$\frac{\partial L}{\partial W^{(1)}} = \frac{\partial L}{\partial \text{pre-act}} \cdot x^T = \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix} \begin{bmatrix} -0.25 & -0.4 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

Gradient w.r.t. hidden bias

$$\frac{\partial L}{\partial b^{(1)}} = \frac{\partial L}{\partial \text{pre-act}} = [\mathbf{0.0}, \mathbf{0.0}]^T$$

Summary Table

Variable	Formula/Computation	Value
L	$-[y_{\text{true}} \log(\hat{y}) + (1 - y_{\text{true}}) \log(1 - \hat{y})]$	0.853

Gradient	Formula (Chain Rule)	Evaluated Value
$\frac{\partial L}{\partial \hat{y}}$	$-\left[\frac{y_{\text{true}}}{\hat{y}} - \frac{1-y_{\text{true}}}{1-\hat{y}}\right]$	2.347
$\frac{\partial L}{\partial \hat{z}}$	$\hat{y} - y_{\text{true}}$	0.574
$\frac{\partial L}{\partial \mathbf{h}}$	$\frac{\partial L}{\partial \hat{z}} \cdot W^{(2)T}$	$[0.459, -0.344]^T$
$\frac{\partial L}{\partial \mathbf{W}^{(2)}}$	$\frac{\partial L}{\partial \hat{z}} \cdot h^T$	$[0.0, 0.0]$
$\frac{\partial L}{\partial b^{(2)}}$	$\frac{\partial L}{\partial \hat{z}} \cdot 1$	0.574
$\frac{\partial L}{\partial \mathbf{W}^{(1)}}$	$\frac{\partial L}{\partial \text{pre-act}} \cdot x^T$	$[[0, 0], [0, 0]]$
$\frac{\partial L}{\partial \mathbf{b}^{(1)}}$	$\frac{\partial L}{\partial \text{pre-act}}$	$[0.0, 0.0]^T$

2.d

i. Spatial Clustering

The embeddings reveal clear geometric clustering by sentiment: positive words ("amazing", "love") cluster in the upper-right quadrant with large positive values on both dimensions, while negative emotion words ("angry", "hate", "scared", "worried") cluster in the lower-left quadrant with consistently negative values. Neutral and function words ("of", "this", "so") cluster near the origin $[0, 0]$, reflecting their lack of emotional content.

ii. Semantic Similarity and Advantage over Bag-of-Words

Word embeddings encode semantic similarity as geometric proximity: words with similar meaning or emotional sentiment are placed close together in vector space, meaning operations like distance and direction carry real language meaning. Unlike bag-of-words, which treats every word as an independent, orthogonal dimension (so "love" and "amazing" share no information), embeddings allow the model to generalize across synonyms and related words: a model that learns "hate" is negative automatically gains partial knowledge about "angry" simply because they occupy similar regions of the embedding space.

3.h

If the learning rate is too small (e.g., $lr = 0.0001$), the loss decreases very slowly (tiny steps mean many epochs needed to converge) and accuracy barely changes.

If the learning rate is too large (e.g., $lr = 100.0$), the loss explodes (overshoots the minimum, bouncing around or diverging entirely). Accuracy becomes zero as the weights become too large.

If the learning rate is just right (e.g., $lr = 0.1$), the loss decreases steadily and accuracy improves consistently until it reaches the max value of 1.0.

5.a

Claude Code: Product User Base and Demographics

Claude (including Claude Code) does not have a fully transparent, product-specific user breakdown, but multiple sources provide a clear overall picture of its user base and demographics.

As of 2025–2026, Claude has roughly 18–30 million monthly active users, with rapid growth (reportedly ~1 million new users per day in early 2026) and strong enterprise adoption across 300,000+ business customers ([Backlinko, 2026](#)). Usage is globally distributed across 150+ countries, led by the U.S. and India, and skewed toward work and coding tasks, with developers and knowledge workers forming a major segment ([GetPanto, 2026](#)). Demographically, users skew young (about 50%+ aged 18–24) and male-dominated (~77%), with a mix of work (~46%), study (~19%), and personal use (~35%) ([DemandSage, 2026](#)).

Overall, while Claude Code-specific user counts are not disclosed, Anthropic reports ~300% growth in its developer user base in 2025, indicating rapid adoption among engineers and enterprise teams ([The New Stack, 2025](#)).

5.b

Claude Code: Intended Use Cases and Audience

Claude Code is designed primarily as an AI-powered coding assistant that enables users to write, refactor, debug, and manage software projects through natural language, often by executing commands directly in the terminal and interacting with entire codebases ([Anthropic GitHub Docs, 2026](#); [Codecademy, 2026](#)). Its core use case is to delegate substantial engineering tasks - such as code generation, documentation, and workflow automation - to an agentic AI system that can operate across files and tools ([DataCamp, 2026](#)).

The primary target users are software developers, engineers, and technical teams, though the tool is increasingly used by adjacent roles (e.g., designers and product builders) to accelerate prototyping and development workflows ([Anthropic GitHub Docs, 2026](#); [DataCamp, 2026](#)).

5.c

Claude Code: Market Share and Competitors

Claude Code operates in the rapidly growing AI coding assistants market, where competition is led by tools such as GitHub Copilot, OpenAI Codex, Cursor, and Google Gemini. While OpenAI still dominates overall AI assistant usage, the coding segment is more fragmented, with estimates suggesting Claude-based tools hold roughly 24–32% market share, comparable to Copilot and Cursor in a three-way tie at the top ([LinkedIn Analysis, 2025](#); [ITiger, 2025](#)).

Some reports indicate even stronger positioning in code generation specifically, with Claude models capturing up to ~42% share among developers, significantly ahead of OpenAI's ~21% ([CB Insights, 2025](#); [Yahoo Finance, 2025](#)). However, broader developer surveys still show tools like ChatGPT and Copilot leading in overall adoption, indicating that Claude Code is a top-tier but not dominant player in a fast-evolving and highly competitive market ([CodeSignal, 2025](#)).

Overall, Claude Code is positioned as a leading contender in the coding-specific AI segment, particularly strong in enterprise and advanced code-generation workflows, but faces intense competition from incumbents and rapidly growing startups in a fragmented market.

5.d

Claude Code: Misuse Risks and Company Safeguards

Claude Code presents a clear misuse risk in that it can be used to generate or modify malicious software, such as automating the creation of exploits, phishing scripts, or obfuscated code that bypasses security controls. Anthropic addresses this through its Responsible Scaling Policy and usage guidelines, which explicitly restrict assistance for malware development,

require safety filtering of harmful outputs, and incorporate monitoring and rate-limiting for suspicious usage patterns ([Anthropic, 2024](#); [Anthropic Acceptable Use Policy, 2025](#)). Additionally, Claude models are trained with safety alignment techniques to refuse or redirect harmful requests, particularly those involving cybersecurity abuse or unauthorized system access ([Anthropic, 2023](#)).

This highlights the dual-use nature of Claude Code: the same capability that enables developers to efficiently write, debug, and secure code can also be repurposed to generate harmful or deceptive software. For example, a beneficial use case is automating code reviews and improving security practices, while a harmful misuse would be accelerating the development of malware or social engineering tools. This duality makes governance challenging, requiring continuous safeguards, policy enforcement, and model-level alignment to balance innovation with risk mitigation.

5.e

Claude Code: Gaps and Unintended Use Cases

Despite existing safeguards, gaps remain in protecting against prompt injection and data exfiltration, where Claude Code can be induced to reveal sensitive files (e.g., API keys) or ignore ignore-rules like `.claudeignore`, indicating incomplete enforcement of access controls ([The Register, 2026](#)). Additionally, real-world misuse cases show that attackers have leveraged Claude to assist in cybercrime activities such as phishing, malware generation, and even large-scale extortion schemes, demonstrating that safeguards can be bypassed in practice ([Reuters, 2025](#); [LessWrong, 2025](#)).

A notable unintended use case is the use of Claude Code as an automated hacking or extortion assistant, where users repurpose its coding and reasoning abilities to scale attacks against organizations - an outcome clearly outside Anthropic's intended productivity - focused design. These examples highlight that, while safeguards exist, agentic coding tools still introduce new attack surfaces and failure modes that are not fully mitigated by current policies or technical controls.